# אלגוריתמים 2: הרצאות $5-6$

7 בנובמבר 2026

**גיא יער־און**

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיהם של פרופסור אלי פורת ופרופסור משה לויינשטיין.
השימוש בסיכום באחריותכם בלבד.

## 1 אלגוריתמי Online

### 1.1 הגדרה

אלגוריתם קלאסי הוא אלגוריתם שמקבל כקלט בעיה, ומוצא לה פתרון. זהו כל עולם האלגוריתמים שהכרנו עד היום. עם זאת, בתרחישים רבים צריך לקבל החלטות בזמן אמת, ובתנאים של אי ודאות. כאן, אנו עוברים לעולם האמיתי והדינמי. בניגוד לאלגוריתמים קלאסיים בהם הכל ידוע מראש, במערכות מסוג זה המערכת צריכה להגיב בזמן אמת לאירועים (Event) שמתקבלים בצורה אקראית או מדורגת, ולייצר תגובות - מבלי לדעת מה יקרה בעתיד. כלומר, אלגוריתם אונליין הוא אלגוריתם בו יש הנחה כי הקלט נחשף לאורך זמן, ולא כל המידע נמסר מראש. האתגר הגדול ביותר שעומד בפנינו באלגוריתמים אונליין הוא כיצד לקבל החלטה מחייבת כרגע, מבלי לדעת מה יהיו הנתונים או הבקשות שיגיעו בעתיד: יתכן שהחלטה שנבצע עכשיו תתגלה כגרועה כשיהיה בידינו את כל הקלט. אם כן, נרצה לשאול את עצמנו: כיצד ניתן למדוד כמה האלגוריתם שלנו טוב ביחס למצב האמיתי?

**הגדרה 1.** אלגוריתם אונליין $\mathcal{A}$ עבור בעיה $P$ הוא $\alpha$־תחרותי אם לכל סדרת קלט $S$ מתקיים אי השוויון הבא:

$$C_{\mathcal{A}}(S) \leq \alpha \cdot C_{OPT}(S) + \beta$$

באשר $C_{\mathcal{A}}$ היא העלות של האסטרטגיה של האלגוריתם $\mathcal{A}$, וכן $C_{OPT}$ היא העלות של האסטרטגיה האופטימלית בהינתן שיודעים את כל הקלט מראש. נעיר כי $OPT$ הוא אלגוריתם שיודע הכל מראש, ואיננו אלגוריתם אונליין.
נשים לב כי הקבוע $\beta$ נחוץ להגדרה, שכן נניח שהעלות האופטימלית קטנה מאוד או אפילו שווה לאפס. אם נרצה לחלק את העלות שלנו בעלות האופטימלי, נקבל מספר עצום - ואז שום אלגוריתם לא יוכל להיות מוגדר כתחרותי. לכן הקבוע $\beta$ נועד בדיוק עבור הצעדים הראשונים וההתחלתיים של האלגוריתם, בהם העלות של האופטמלי קטנה מאוד או אפסית - והוא מונע מהיחס להתפוצץ בגלל אותם מקרים.

### 1.2 בעיית Path Search

#### 1.2.1 נסיון ראשון: אלגוריתם לא תחרותי

נניח כי דוד מקדונלד עומד בצומת - ממנה מתפצלים שני כבישים אינסופיים. הוא צריך למצוא את הפרה שמונחת באחד מהם, מבלי לדעת באיזה אחד מהם היא בחרה. על מנת לפתור את זה, נניח אסטרטגיה לפתרון של דוד מקדונלד: אסטרטגיה דמוית $BFS$ - הרעיון הוא לחפש בצורה סימטרית ושקולה בין שני הכבישים, האלגוריתם מבצע את הצעדים הבאים:

$$(+1, -1, +2, -2, +3, -3, ....., +i, -i, ...)$$

כלומר: הולכים ימינה עד מרחק 1, ואם לא מצאנו: חוזרים לאמצע. הולכים שמאלה עד מרחק אחד, ואם לא מצאנו - חוזרים לאמצע. הולכים ימינה עד מרחק 2, אם לא מצאנו: חוזרים לאמצע. וכן הלאה. בכל פעם מגדילים את טווח החיפוש $i$ באחד.

כעת, נרצה לראות מדוע האלגוריתם שהצענו **איננו** אלגוריתם תחרותי. ובכן, נניח לצורך הניתוח כי הפרה נמצאת בה"כ בצד השמאלי במיקום מסוים $-t$. כלומר, $t$ צעדים מהאמצע שמאלה.
האלגוריתם האופטימלי, יודע שהפרה נמצאת במיקום $-t$. הוא לא מבזבז זמן, והולך לשם ישירות ולכן:

$$C_{OPT}\left(-t\right)=t$$

האלגוריתם שלנו לעומת זאת, הולך וחוזר שוב ושוב ועלותו:

$$C_{\mathcal{A}}\left(-t\right)=2\cdot\left(1+1+2+2+....+\left(t-1\right)+\left(t-1\right)+t\right)+t\geq4\sum_{i=1}^{t-1}i=\Omega\left(t^{2}\right)$$

ולכן:

$$\frac{C_{\mathcal{A}}\left(-t\right)}{C_{OPT}\left(-t\right)}=\frac{\Omega\left(t^{2}\right)}{t}=\Omega\left(t\right)\rightarrow_{t\rightarrow\infty}\infty$$

כלומר היחס הנ"ל איננו נחסם בקבוע כלשהו, ולכן האלגוריתם הנ"ל איננו תחרותי.

#### 1.2.2 נסיון שני: אלגוריתם גאומטרי

כעתת במקום להגדיל את צעדי החיפוש באחד בכל פעם, נגדיל את צעדי החיפוש בצורה גאומטרית - לפי חזקות של 2.

**משפט 2.** קיים אלגוריתם ביחס תחרות 13 ל*Path Search*.
הוכחה: נתבונן באסטרטגיית הצעדים

$$\left(+1,-1,+2,-2,+4,-4,.....,+2^{i},-2^{i},...\right)$$

כיצד נראה המקרה הגרוע ביותר? נניח כי הפרה נמצאת במקום השלילי הגרוע ביותר האפשרי לפני שנגלה אותו בטווח הבא, כלומר $-\left(2^{n}+\varepsilon\right)$ באשר $2^n$ הוא הטווח בו חיפשנו לפני כן ו$\varepsilon$ פס' חיובי - היא מונחת ממש קצת אחרי $2^n$ ולכן פספסנו אותו.
ראשית, עלות המקרה האופטימלי היא $C_{OPT}\left(-\left(2^{n}+\varepsilon\right)\right)=2^{n}+\varepsilon$ כי הוא יודע היכן הפרה נמצאת. לעומת זאת, עלות האלגוריתם שלנו:

$$C_{\mathcal{A}_{2}}\left(-\left(2^{n}+\varepsilon\right)\right)=2\cdot\left(1+1+2+2+4+4+....+2^{n}+2^{n}+2^{n+1}\right)+2^{n}+\varepsilon=13\cdot2^{n}-4+\varepsilon$$

ולכן:

$$\frac{C_{\mathcal{A}_{2}}\left(-\left(2^{n}+\varepsilon\right)\right)}{C_{OPT}\left(-\left(2^{n}+\varepsilon\right)\right)}=\frac{13\cdot2^{n}-4+\varepsilon}{2^{n}+\varepsilon}\rightarrow13$$

כלומר, אלגוריתם ההליכה הגאופטרי הוא 13־תחרותי.

**משפט 3.** קיים אלגוריתם ביחס תחרות 9 ל$Path\ Search$.
הוכחה: נתבונן כעת באסטרטגיה חכמה יותר, ע"י שינוי סדר הצעדים נצליח להקטין את הקבוע. במקום לחפש תמיד ימינה ואז שמאלה באופן סימטרי, נשתמש בסדר צעדים שמתחיל ישר לכיוון אחד ואז מתהפך.

$$\left(-1,+2,-4,+8,.....,(-1)^i 2^i\right)$$

כיצד נראה המקרה הגרוע ביותר? נניח כי הפרה נמצאת במקום השלילי הגרוע ביותר האפשרי לפני שנגלה אותו בטווח הבא, כלומר $(-1)^n(2^n+\varepsilon)$ באשר $2^n$ הוא הטווח בו חיפשנו לפני כן ו$\varepsilon$ מס' חיובי כך שהיא מונחת ממש קצת אחרי $2^n$ ולכן פספסנו אותו. ראשית, עלות המקרה האופטימלי היא $C_{OPT}\left((-1)^n(2^n+\varepsilon)\right)=2^n+\varepsilon$ כי הוא יודע היכן הפרה נמצאת. לעומת זאת, עלות האלגוריתם שלנו:

$$C_{\mathcal{A}_3}\left((-1)^n(2^n+\varepsilon)\right)=2\cdot\left(1+2+4+....+2^n+2^{n+1}\right)+2^n+\varepsilon=9\cdot 2^n-2+\varepsilon$$

ולכן:

$$\frac{C_{\mathcal{A}_3}\left(-(2^n+\varepsilon)\right)}{C_{OPT}\left(-(2^n+\varepsilon)\right)}=\frac{9\cdot 2^n-2+\varepsilon}{2^n+\varepsilon}\to 9$$

כלומר, אלגוריתם ההליכה הגאומטרי הוא 9־תחרותי.

## 1.3 Ski Rental Problem

הבעיה פשוטה. אדם רוצה לצאת לחופשת סקי ־ הוא עומד בפני שאלה: האם לקנות ציוד או לשכור בכל פעם מחדש?
נתון כי:
- עלות ההשכרה של מגלשת סקי ליום אחד הוא $1.
- עלות הקנייה של מגלשי סקי חדשים היא $M.

הבעיה: האדם יודע שהוא הולך לסקי במשך $t$ ימים סה"כ, אך המספר הזה לא ידוע מראש ־ כלומר זהו אלגוריתם אונליין אמיתי בו היום נגמר והאדם מחליט אם להמשיך את החופשה או לשוב הביתה.
נשים לב כי הפונקציה האופטימלית באופן טריוואלי היא

$$opt(t)=\min\{t,M\}$$

כלומר, בהינתן שאכן ידענו כמה ימים נגלוש ־ היינו בוחרים את האפשרות הזולה מבין השניים. נתבונן באלגוריתם הבא:

| **Algorithm: Ski Rental Problem** |
|---|
| 1. Rent for $M$ days, then buy |

כלומר, שוכרים את המגלשיים במשך $M$ ימים. בעת שמגיעים ליום ה$M$, קונים אותם.

**משפט 4.** מדובר באלגוריתם 2־תחרותי.
הוכחה: נחלק למקרים.
מקרה ראשון: מס' הימים קטן מעלות הקנייה. כלומר, $t < M$. אם כן במקרה שלנו שילמנו במשך $t$ ימים להשכרה ולכן שילמנו $t$ דולרים. ובכן, נראה כי כיוון ש$t < M$ מתקיים

$$C_{OPT}(t) = \min\{t, M\} = t$$

ולכן:

$$\frac{C_{\mathcal{A}}(t)}{C_{OPT}(t)} = \frac{t}{t} = 1 \leq 2$$

מקרה שני: מס' הימים גדול או שווה מעלות הקנייה. כלומר $t \geq M$. אם כן, שכרנו בדיוק במשך $M$ ימים ואז שילמנו עוד $M$ דולרים ולכן סה"כ $C_{\mathcal{A}}(t) = 2M$. וכן מתקיים

$$C_{OPT}(t) = \min\{t, M\} = M$$

לכן:

$$\frac{C_{\mathcal{A}}(t)}{C_{OPT}(t)} = \frac{2M}{M} = 2$$

**משפט 5.** כל אלגוריתם דטרמיניסטי עבור בעיית השכרת הסקי הוא לפחות 2־תחרותי.
הוכחה: נניח אלגוריתם דטרמיניסטי שבוחר לקנות לאחר זמן כלשהו $s$. אם נבחר בדיוק קלט עבורו $t = s$ העלות שלנו יהיה השכרה עד לנקודה זו ועוד קנייה: $s + M$. בעוד שהאופטימלי היה משלם $\min\{s, M\}$. מכאן מתקיים

$$\frac{C_{\mathcal{A}}(t)}{C_{OPT}(t)} = \frac{s+M}{\min\{M,s\}} = \frac{s}{\min\{M,s\}} + \frac{M}{\min\{M,s\}} \geq 1 + 1 = 2$$

כלומר, באמצעות כל אלגוריתם דטרמיניסטי לא ניתן להשיג תוצאה טובה מ2 יחס קירוב. נשים לב כי יחס תחרותי מוגדר לפי המקרה הגרוע ביותר; כיוון שהצגנו מקרה שבו יחס התחרות גדול או שווה מ2, בהכרח המקרה הגרוע ביותר מניב יחס תחרות לפחות 2, ולכן יחס התחרות לפחות 2.

ובכן, לאור המגבלה הזו. זה יותר ממתבקש שנתחיל לדון באלגוריתמי תחרות רנדומיים.

**הגדרה 6.** אלגוריתם אקראי מוגדר כפונקציה שלוקחת מחרוזת ביטים אקראיים $R \in \{0,1\}^*$ וממפה אותם לאלגוריתם דטרמיניסטי $\mathcal{A}_R$.
יריב עיוור מוגדר להיות כזה שמכיר את קוד האלגוריתם ואת התפלגות הבחירות האפשריות, אך איננו יודע אילו ביטים אקראיים נבחרו בפועל עבור ריצה מסוימת.

**הגדרה 7.** אלגוריתם אקראי $\mathcal{A}$ הוא $\alpha$־תחרותי מול יריב עיוור אם לכל קלט קבוע $S$:

$$\mathbb{E}_{R\sim\mathcal{D}}[C_{\mathcal{A}_R}(S)] \leq \alpha \cdot C_{OPT}(S) + \beta$$

באשר $\beta$ קבוע אבסולוטי. כלומר, התוחלת של ביצועי האלגוריתם שלנו על קבוע חסומה ב$\alpha$ כפול העלות האופטימלית.

ראשית, נתבונן באלגוריתם הסתברותי ראשון:

**Algorithm 1: Randomized Ski Rental**

1. With probability $\frac{1}{2}$, buy at time $\frac{M}{2}$; with probability $\frac{1}{2}$, buy at time $M$

כלומר, קנה בזמן $\frac{M}{2}$ בהסתברות חצי, ובהסתברות חצי קנה בזמן $M$.
נתבונן במקרה הגרוע ביותר. זהו המקרה שבו התהליך נעצר מיד לאחר שהאלגוריתם מקבל החלטה לקנות - למעשה בזבוז של הכסף לקנייה. באשר $s = M$ נקבל

$$\mathbb{E}[C(M)] = \frac{1}{2} \cdot \left(\frac{M}{2} + M\right) + \frac{1}{2}(M + M) = \frac{7}{4}M = \frac{7}{4}C_{OPT}(M)$$

עבור $s = \frac{M}{2}$ נקבל:

$$\mathbb{E}[C(M)] = \frac{1}{2}\left(\frac{1}{2}M + M\right) + \frac{1}{2}\left(\frac{M}{2}\right) = M = 2 \cdot C_{OPT}(M)$$

כלומר, אם מס' הימים הוא $\frac{M}{2}$ נקבל אותו יחס תחרות כמו באלגוריתם הדטרמיניסטי. וזה כמובן, לא טוב. לכן נתבונן באלגוריתם טוב יותר:

**Algorithm 2: Randomized Ski Rental**

1. With probability $\frac{1}{2}$, buy at time $\alpha M$; with probability $\frac{1}{2}$, buy at time $M$

כעת, בהסתברות חצי קונים בזמן $\alpha M$ ובהסתברות חצי קונים בזמן $M$. כאן $\alpha \in [0, 1]$.
באשר $s = M$ נקבל

$$\mathbb{E}[C(M)] = \frac{1}{2} \cdot (\alpha M + M) + \frac{1}{2}(M + M) = \left(\frac{\alpha + 3}{2}\right) M = \left(\frac{\alpha + 3}{2}\right) C_{OPT}(M)$$

עבור $s = \alpha M$ נקבל:

$$\mathbb{E}[C(M)] = \frac{1}{2}(\alpha M + M) + \frac{1}{2}(\alpha M) = \frac{2\alpha + 1}{2}M = \left(1 + \frac{1}{2\alpha}\right) \cdot C_{OPT}(M)$$

שהרי המעבר האחרון נכון כי האופטימלי במקרה זה הוא $\alpha$.
אם כן, היחס התחרותי הינו המקסימום בין שני המקרים. כלומר:

$$\max\left\{\frac{\alpha + 3}{2}, 1 + \frac{1}{2\alpha}\right\}$$

$$\implies 1 + \frac{1}{2\alpha} = \frac{\alpha + 3}{2} \implies \alpha = \frac{\sqrt{5} - 1}{2} \approx 0.618$$

ולכן יחס הקירוב הינו

$$\frac{\frac{\sqrt{5}-1}{2} + 3}{2} = \frac{\sqrt{5} + 5}{4} \approx 1.809$$

כלומר, שיפור ביחס לגרסה הדטרמיניסטית.

## Paging 1.4

נזכר קצת במערכות הפעלה, ונדון כעת באלגוריתמי Paging שכבר למדנו. נניח מערכת עם זוג רמות זיכרון:
- זיכרון ראשי: מורכב מ $k$ PAGES , קצב ההעברה בו מהיר מאוד.
- דיסק קשיח: זיכרון חיצוני שמכיל מס' אינסופי של עמודים, אך איטי משמעותית.

המטרה: בכל פעם שמבקשים $PAGE$ שלא בזיכרון הראשי ונוצר Page Fault, צריך להביא אותו מהדיסק ולפנות עמוד קיים מהזיכרון הראשי על מנת לפנות לו מקום. המטרה היא לצמצם למינימום את מספר הPage Faults.

בניגוד למערכות הפעלה, נניח כי בהינתן דוגמה עם $k = 3$ וסדר בקשות לדפים $(1, 2, 3)$ יתרחשו 0 Page Faults, שכן לא נדרשנו להחליף.

### אלגוריתם אופטימלי 1.4.1

האלגוריתם הפשוט פועל כך:

**Optimal Algorithm: Paging**

1. Replace the page that will be needed in the most distant future

כלומר, זהו לא אלגוריתם Online אלא אלגוריתם Offline שמניח שסדרת הבקשות העתידית ידועה מראש. האסטרטגיה הפשוטה היא להחליף תמיד את הPage שיהיה בו צורך בזמן הכי רחוק בעתיד - למרות שמדובר באלגוריתם Offline שלא ניתן ליישם במציאות, הוא ישמש כנקודת ייחוס להשוואה לשאר האלגוריתמים. כמובן, שהוא הכי טוב וממזער את מס' הPage Faults.

### Least recently used (LRU) 1.4.2

האלגוריתם הוא אלגוריתם Online שמוגדר באופן הבא - כאשר מתרחש Page Fault, האלגוריתם מוציא מהזיכרון את העמוד שעבר הזמן הרב ביותר מאז השימוש האחרון בו.

**Algorithm: Least Recently Used (LRU)**

1. Upon a page fault, remove the page which was last used farthest in the past

**משפט 8.** אלגוריתם ה$LRU$ עבור זיכרון מטמון בגודל $k$ הוא $k$־תחרותי מול יריב *Offline*. כלומר:

$$LRU\left(\sigma\right) \leq k \cdot Opt + \mathcal{O}\left(1\right)$$

לכל סדרת בקשות $\sigma$ .
הוכחה: תהי סדרת בקשות, $\sigma = p_1, ....., $. נחלק את הסדרה לבלוקים זרים סדורים $\sigma_1, ...., \sigma_t$ כך שלכל בלוק $\sigma_i$ הוא תת סדרה מקסימלית המכילה בדיוק $k$ עמודים שונים.
ראשית, בכל בלוק $\sigma_i$ אלגוריתם ה$LRU$ חווה לכל היותר $Page\ Faults\ k$, שכן בתוך בלוק ישנם סה"כ $k$ עמודים שונים שנדרשים, לכן בסקרה הגרוע ביותר כל עמוד שונה שלא נמצא בזיכרון יגרום סה"כ לשגיאה אחת. אם ישנם סה"כ $t$ בלוקים כנ"ל נקבל חסם של $LRU$ של:

$$LRU \leq k \cdot t$$

על מנת להוכיח כמה שגיאות האלגוריתם האופטימלי חייב לבצע, נגדיר את $P_i$ להיות העמוד הראשון בבלוק $\sigma_i$. נגדיר בלוק מוסט באופן הבא:

$$\sigma_i' = (\sigma_i \backslash P_i) \cup P_{i+1}$$

כלומר, אנו מוריזים את העמוד הראשון, ומוסיפים את העמוד הראשון בבלוק הבא. נשים לב - $Opt$ חייב להכיל שגיאה אחת לפחות בכל בלוק מוסט. שכן, לפי הגדרת המקסימליות של בלוק $\sigma_i$, העמוד הבא $P_{i+1}$ שפותח את הבלוק הבא, אינו שייך לקבוצת $k$ העמודים השונים של $\sigma_i$. לכן, הבלוק המוסט $\sigma_i'$ מכיל סה"כ לפחות $k+1$ עמודים שונים.
כיוון שלזיכרון המטמון של $Opt$ יש קיבולת מקסימלית של $k$ עמודים, בלוק המכיל $k+1$ עמודים מחייב את אלגוריתם $Opt$ לבצע לפחות שגיאת עמוד אחת בכל בלוק מוסט $\sigma_i'$. לכן, $Opt \geq t$. מכאן נקבל:

$$\frac{LRU}{OPT} \leq \frac{LRU}{t} \leq \frac{k \cdot t}{t} = k$$

ולכן אלגוריתם $LRU$ הוא $k$־תחרותי.

הערה 9. ביחס להוכחה לעיל: נסביר מה הכוונה בחלוקה לסדרות רציפות מקסימליות. נניח $k = 3$. נניח רצף $(A, B, C, B, D, E, F)$. $4$ הראשונים ברצף מכילים $3$ עמודים ולכן $\sigma_1 = (A, B, C, B)$. לאחר מכן נקבל כי $\sigma_2 = (D, E, F)$ - אלו $3$ עמודים בדיוק. מה הבעיה? אנחנו לא יכולים לחסום על רצפים כאלו את מס' השגיאות. אלגוריתם $OPT$ כרגע יתכן ולא יטעה כלל. לכן, אם נרצה להכריח אותו לעשות שגיאה מבחינת הניתוח שלנו - משתמשים בעקרון שובך היונים: יש לו מקום ל$k$ עמודים, לכן נשנה את ההסתכלות שלנו לחלונות בגודל $k+1$ עמודים. שם - בוודאות תתרחש שגיאה.

**משפט 10.** חסם תחתון לכל אלגוריתם דטרמיניסטי לבעיית ה*Paging*. כל אלגוריתם *PAGING* דטרמיניסטי $A$ הוא לפחות $k$־תחרותי, כלומר קיים יחס תחרות המקיים

$$\frac{C_A(\sigma)}{C_{OPT}(\sigma)} \geq k$$

עבור סדרת בקשות מתאימה $\sigma$ באורך גדול מספיק.
הוכחה: נניח כי מרחב העמודים האפשרי הכולל הוא $n = k+1$, כלומר ישנו בדיוק עמוד אחד יותר מקיבולת זיכרון המטמון. יהי $A$ אלגוריתם דטרמיניסטי כללי כלשהו.
היריב מייצר סדרת בקשות אונליין באופן הבא: בכל שלב, היריב מבקש את העמוד היחיד שלא נמצא כרגע בזיכרון של האלגוריתם $A$.
כיוון ש$A$ הוא אלגוריתם דטרמיניסטי, התגובה שלו לכל בקשה נקבעה מראש. כיוון שהעמוד המבוקש הוא תמיד בדיוק זה שנעדר מהזיכרון של $A$, אלגוריתם $A$ יחווה *Page Fault* ב$100\%$ מהבקשות. אם סדרת הבקשות באורך $N$ נקבל כי

$$C_A(\sigma) = N$$

כעת, נתבונן במקום באלגוריתם האופטימלי. נזכר כי האופטימלי הוא *Belady's Algorithm*, אלגוריתם שמסלק מהזיכרון בכל שלב את העמוד שיהיה בו צורך הכי רחוק בעתיד. כאשר מתרחש *Page Fault* אצל $OPT$, הוא מכניס לזיכרון את העמוד המבוקש. כיוון שישנם סה"כ $k+1$ עמודים אפשריים, והרי $OPT$ שומר בזיכרון שלו $k$ עמודים, ברגע שהוא נאלץ להוציא עמוד הוא מוציא את זה שלא יופיע בזמן הקרוב יותר. לכן לאחר כל שגיאת עמוד, $OPT$ מצליח להחזיק בזיכרון את כל $k-1$ הבקשות שיגיעו ולכן הוא סובל משגיאה אחת בלבד מתוך כל מחזור של $k$ בקשות. לכן נקבל כי על פני $N$ בקשות מתקיים

$$C_A(\sigma) \leq \frac{N}{k}$$

לכן נקבל:

$$\frac{C_A(\sigma)}{C_{OPT}(\sigma)} \geq \frac{N}{\frac{N}{k}} = k$$

ולכן לכל אלגוריתם דטרמיניסטי כלשהו $A$, יחס התחרות ביחס לפתרון האופטימלי הוא לפחות $k$. נעיר כי מספיק להסתכל על מקרה אחד, כי יחס תחרות נקבע ביחס למצב הגרוע ביותר.

## 1.5 Random Paging

ובכן, כפי שהוכחנו כל אלגוריתם דטרמיניסטי מקיים יחס תחרות של לפחות $k$, והרי LRU מקיים יחס תחרות של בדיוק $k$, מדובר באלגוריתם Paging הדטרמיניסטי הטוב ביותר שניתן למצוא. אם ככה, ברור שנתפנה לאלגוריתמי Paging רנדומיים.
באופן נאיבי - למה שלא נציע את האלגוריתם הבא: בעת Page Fault־ בחר עמוד אקראי, והוצא אותו מהזיכרון.
ובכן, זה רעיון רע מאוד. הוא לא ישפר דבר ביחס לאלגוריתמים הדטרמיניסטיים. נניח שישנם $n = k+1$ עמודים אפשריים, נניח שזיכרון המטמון ההתחלתי של האלגוריתם ושל האלגוריתם האופטימלי מכיל את העמודים:

$$\{1, ..., k\}$$

נגדיר פרמטר $m >> k$ ונבנה את סדרת הבקשות הבאה:

$$\sigma = (k+1, 1, 2, , .....k-1)^m (k, 1, 2, .....k-1)^m$$

באשר למקטע הבקשות הראשון נקרא $A$ ולשני $B$. אלגוריתם OPT האופטימלי יודע את העתיד - לכן הוא בכל פעם זורק את העמוד שלא יהיה בשימוש זמן רב, לכן הוא מחליף רק בין $k$ ל$k+1$. לכן הוא מייצר שגיאה אחת לכל $k \cdot m$ בקשות.
לעומת זאת, האלגוריתם הנאיבי הרנדומי, צריך לזכור באקראי. ההסתברות שהוא יזרוק את העמוד הנכון - זה שהוא לא ישתמש בו בזמן הקרוב במקטע הארוך היא $\frac{1}{k}$, אם יזרוק עמוד שגוי: הוא מיד יחווה שגיאה נוספת בבקשה הבאה - כיוון שהמקטע ארוך

מאוד באורך $m$, ניתן להוכיח שתוחלת השגיאות בכל מקטע היא לפחות $k$. לכן, אשר $m$ שואף לאינסוף יחס התחרות ישאף לאנסוף. שכן הוא בדיוק חסום ב $\frac{mk}{k} = m$.

### 1.5.1 Random Marking Algorithm - RMA

כעת נראה אלגוריתם שמאפשר לשבור את יחס $k$־התחרות ולהגיע ליחס תחרות הרמוני. נתבונן באלגוריתם הבא:

**Algorithm: Random Marking Algorithm (RMA)**

1. Run in "phases":
   (a) At the beginning of a phase, all pages are unmarked
   (b) Every page requested becomes marked (whether it is a page fault (PF) or not)
   (c) Upon a PF, discard a random unmarked page
   (d) On a PF, if all the pages are marked, begin a new phase (unmark all)

האלגוריתם עובד בשלבים ־ Phases: בתחילת כל שלב, כל העמודים בזיכרון מסומנים כלא־מסומנים. במהלך השלב, כל עמוד שמתקבל בבקשה ( בין אם הוא גורם לשגיאה או שלא ) הופך למסומן. בעת Page Fault, האלגוריתם מסלק מזיכרון המטמון עמוד לא מסומן באקראי מתוך הזיכרון. המעבר לשלב הבא מתרחש: אם מתרחשת PF ובזיכרון אין בכלל עמודים לא מסומנים ־ כלומר אין את מי לסלק, מתחיל שלב חדש: כל הסימונים מאופסים חזרה, והתהליך חוזר על עצמו.

נשים לב כי גבולות השלבים לא מושפעים מההחלטות האקראיות של האלגוריתם, אלא נקבעים בצורה דטרמיניסטית לחלוטין ע"י סדרת הבקשות. למעשה, כל שלב הוא רצף מקסימלי של בקשות המכיל בדיוק $k$ עמודים שונים.

**משפט 11.** אלגוריתם $RMA$ הוא אלגוריתם $2H_k$ תחרותי מול יריב, כך ש$H_k = \sum_{j=1}^{k} \frac{1}{j}$.
הוכחה:
נתחיל בחסם על מס' השגיאות של האלגוריתם ב$Phase$ בודד. בתוך שלב נתון $i$, אלגוריתם ה$RMA$ גובה לפחות $m_i$ שגיאות וודאיות באשר $m_i$ הוא מס' העמודים החדשים שהופיעו בשלב זה ביחס לשלב הקודם ונדרשו ל$PF$. על מנת להבין כמה שגיאות נוספות צפויות להיות עקב הבחירה האקראית, נתבונן במבנה הזיכרון ברגע נתון בתוך השלב:
הזיכרון בגודל $k$ מתחלק לשלושה סוגים של עמודים:
1. $l$ עמודים חדשים מסומנים: עמודים שהופיעו לראשונה בשלב זה וכבר טופלו.
2. $j$ עמודים ישנים מסומנים: עמודים מהשלב הקודם שכבר התבקשו והוכתרו כקבועים בשלב הנוכחי ולא גרמו ל$PF$.
3. $k - j - l$ עמודים ישנים לא מסומנים: עמודים מהשלב הקודם שטרם ביקשו אותם בשלב הנוכחי ונשארו בזיכרון.
באשר מגיעה בקשה לעמוד ישן שטרם נראה בשלב הנוכחי, ההסתברות לשגיאת עמוד תלויה בעמודים הלא מסומנים שנותרו בזיכרון. ההסתברות לשגיאה זו:

$$Pr(PF) = \frac{l}{k-j} \leq \frac{m_i}{k-j}$$

שכן נשים לב כי האלגוריתם יכול לקבל שגיאות עמוד משני גורמים: עמודים חדשים בשלב הזה שלא הופיעו בקודם, או עמודים ישנים - אבל נשים לב כי עמוד ישן יגרום לשגיאה רק אם הוא הוצא בעבר ע"י האלגוריתם בבחירה אקראית כלשהי.
אם נסכום את התוחלת של שגיאות העמוד על פני כל השלב ה$i$ נקבל כי:

$$\mathbb{E}[PF] = m_i + \sum_{j=0}^{k-m_i-1} \frac{m_i}{k-j} \leq m_i \cdot \sum_{j=0}^{k} \frac{1}{j} = H_k \cdot m_i$$

כעת, נרצה לנתח את האלגוריתם האופטימלי. לכן נתבונן על זוגות $(i-1, i)$ עוקבים. בשילוב של שני שלבים אלו, נדרשים בסך הכל לפחות $k + m_i$ עמודים שונים, מה שמחייב את אלגוריתם $OPT$ לבצע לפחות $m_i$ שגיאות עמוד. לכן נקבל כי סה"כ מס' שגיאות העמוד הס

$$OPT \geq \frac{1}{2} \sum_{i \geq 2} m_i$$

סה"כ נראה כי

$$\mathbb{E}(RMA) \leq H_k \cdot \sum_i m_i \leq 2H_k \cdot OPT$$

לכן יחס התחרות המפוצץ של האלגוריתם הוא $2H_k$.

**משפט 12.** ([$McGeoch, Sleator 1991$]) כל אלגוריתם רנדומי עבור $Paging$ עם יחס תחרות של לפחות $H_k$ נגד יריב, וקיים אלגוריתם $Paging$ רנדומי עם יחס תחרות $H_k$.

## 1.6 בעיית המזכירה

**הגדרה 13.** בעיית המזכירה מוגדרת באופן הבא: יש בידינו משרה אחת זמינה אותה נרצה לאייש במועמד המוצלח ביותר. ידוע לנו מראש כי מס' המועמדים שיגיעו להתראיין הוא $n$. לאחר שמתראיין מועמד, אנו יודעים לדרג אותו באופן יחסי אך ורק ביחס לכל המועמדים שהתראיינו באותו רגע. ההחלטה האם לקבל או לדחות מועמד: חייבת להתקבל מיד בסיום הראיון שלו. אם דחינו מועמד - אין אפשרות לחזור אליו בהמשך. המטרה: נצליח במשימה אם ורק אם בחרנו את המועמד הטוב ביותר מכולם, התשלום הוא 1 אם בחרנו את הטוב ביותר ו0 אחרת. כמו כן, המועמדים מגיעים לראיונות בסדר אקראי אחיד. מטרתנו: מציאת אסטרטגיה חכמה שתמקסם את ההסתברות לבחירת המועמד הטוב ביותר.

### 1.6.1 אלגוריתם ראשון

נתבונן באלגוריתם הבא:

**Algorithm: Secretary Problem**

1. Ignore the first $\frac{n}{2}$ applicants
2. Pick the first candidate among the last $\frac{n}{2}$ candidates which is better than all the first $\frac{n}{2}$ candidates

האלגוריתם מתעלם מ$\frac{n}{2}$ המועמדים הראשונים. הוא בוחר את המועמד הראשון מבין $\frac{n}{2}$ המועמדים האחרונים שהוא טוב יותר מ$\frac{n}{2}$ המועמדים הראשונים שראינו קודם.
נרצה להוכיח חסם תחתון על ההסתברות לבחירת המועמד הטוב ביותר. נגדיר זוג מאורעות:
$\Psi_1$: המאורע שהמועמד הטוב ביותר נמצא בחצי השני של המועמדים.
$\Psi_2$: המאורע שהמועמד השני הכי טוב ממוקם באחד מ$\frac{n}{2}$ המקומות הראשונים.
נטען כי:

$$\Pr[\text{Picking the best}] \geq \Pr[\Psi_1 \wedge \Psi_2]$$

שכן אם אכן המועמד הטוב ביותר נמצא בחצי השני של המועמדים, והשני הכי טוב נמצא ב$\frac{n}{2}$ הראשונים, האלגוריתם יצליח לתפוס את הכי טוב. מכאן:

$$\Pr[\text{Picking the best}] \geq \Pr[\Psi_1 \wedge \Psi_2] = \Pr[\Psi_1] \cdot \Pr[\Psi_2|\Psi_1] = \frac{1}{2} \cdot \frac{\frac{n}{2}}{n-1} \geq \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$$

### 1.6.2 אלגוריתם משופר

כעת, נתבונן באלגוריתם הבא. שהוא כמעט זהה לקודם:

**Algorithm: Secretary Problem (General $\tau$)**

1. Ignore the first $\tau$ applicants
2. Pick the first candidate among the last $n-\tau$ candidates which is better than all the first $\tau$ candidates

כעת, נקח את אותו אלגוריתם ופשוט נדלג על $\tau$ המועמדים הראשונים. לאחר מכן האלגוריתם בוחר את המועמד הראשון ביחס ל$n-\tau$ הנותרים שהכי טוב ביחס לכל ה$\tau$ הראשונים. נראה כי לפי אותו חישוב נקבל כי:

$$\Pr[\text{Picking the best}] \geq \Pr[\Psi_1 \wedge \Psi_2] = \frac{n-\tau}{n} \cdot \frac{\tau}{n-1} \geq \frac{\tau\,(n-\tau)}{n^2}$$

למעשה זהו פרבולה שמקסימלית כאשר $\tau = \frac{n}{2}$. ולמה זה מעניין אותנו? כי הגישה הנאיבית ההסתברותית שלנו לא עובדת. כלומר, החסם שלנו חלש מדי. כעת נרצה לנתח בצורה טובה יותר את החסם.

אנו רוצים לחשב את ההסתברות לבחור את הטוב ביותר. נחלק למקרים לפי המקום הטוב ביותר $t$. המיקום הטוב ביותר חייב להמצא במיקום $t \geq \tau + 1$. מה התנאים לכך שנבחר בו במיקום ה$t$?
- המועמד במיקום $t$ הוא אכן הטוב ביותר גלובלית, ההסתברות לכך היא $\frac{1}{n}$ כי מדובר בסדר אקראי.
- לא נעצור על אף מועמד בין $\tau+1 \to t-1$. על מנת שלא נעצור לפני $t$, ההכי טוב חייב ליפול בטווח של ה$\tau$ הראשונים. אחרת, הוא היה נופל בטווח $[\tau+1, t-1]$ והיינו בוחרים אותו ועוצרים מוקדם מדי. לכן נרצה שהכי טוב יהיה ב$\tau$ הראשונים, ההסתברות

שהכי טוב מבין $t-1$ ראשונים יהיה ב$\tau$ הראשונים היא: $\frac{\tau}{t-1}$. לכן נקבל:

$$\Pr\left[\text{Picking the best}\right] = \sum_{t=\tau+1}^{n} \Pr\left[t^{th} \text{ is global best}\right] \cdot \Pr\left[\text{best of t-1 is in first } \tau\right] = \sum_{t=\tau+1}^{n} \frac{1}{n} \cdot \frac{\tau}{t-1}$$

$$= \frac{\tau}{n} \cdot \left(\frac{1}{\tau} + \frac{1}{\tau+1} + ..... + \frac{1}{n-1}\right) = \frac{\tau}{n} \cdot (H_{n-1} - H_{\tau-1}) \approx \frac{\tau}{n} \cdot \ln\left(\frac{n}{\tau}\right)$$

המקסימום מתקבל ע"י גזירה להיות $\tau = \frac{n}{e}$ ואז מקבלים כי:

$$\Pr\left[\text{Picking the best}\right] \approx \frac{1}{e}$$

הערה 14. דרך החישוב היא למעשה להניח שתמיד יתכן ונטעה, אם הכי טוב הוא ב$\tau$ הראשונים. אחרת: נעבור על כל המקרים עבור $t \in [\tau+1, n]$ ונניח לרגע כי ה$t$ הנ"ל הוא אכן הטוב ביותר גלובלית, ונחשב את הסיכוי שהאלגוריתם אכן יגיד שהוא הכי טוב גלובלית.

0>0